Zagazig University — Faculty of Engineering
Electronics and Communications Engineering Department
ECE 435 — Remote Sensing

Land Use and Land Cover Classification

Using Landsat 8 Multispectral Imagery and Machine Learning

Group Number	[7]
Course	ECE 435 — Remote Sensing
Academic Year	2025 / 2026
Submission Date	19 May 2026

Group Members

#	Student Name
1	**Omnia ELsepaay maraey ELsepaay**
2	**Omnia Mostafa Elshaprawey Kamel**
3	**Tasneem Abdebaset Rashaad Mahmoud**
4	**Raghda Nasr Mohammed Ahmed**
5	**Nadeen Abdo Mohammed Ebrahim**
6	**Nada Atiya Abdelazez Atiya**
7	**Nada Mahmoud Said Helal**
8	**Nour Eldin Amr Mostafa Nasrat**
9	**Youssef Ahmed Mohammed Lotfy**

. Team Contributions

Team Member(s)	Contribution
Nour Amr & Youssef Ahmed	Development of the Training Pipeline and Web Application.
Omnia Moustafa, Omnia Elsepaay, Raghda Nasr, Nada Mahmoud & Nadeen Abdo	Satellite Imagery Processing using ENVI Software.
Omnia Moustafa, Omnia Elsepaay, Raghda Nasr & Nada Mahmoud	Development and implementation of Python Testing Scripts.
Nada Atiya & Tasneem Abdelbaset	Technical Report Writing and Data Collection.
Omnia Moustafa	Presentation Design and Preparation.

Abstract

This report documents the full development and implementation of a land cover classification system for Egyptian satellite imagery using Landsat 8 multispectral data and supervised machine learning. The project covers three distinct components: a training pipeline that processes labeled satellite data and builds a classification model, a standalone testing script that applies the trained model to new images, and a web-based application that allows any user to upload a satellite image and instantly receive a color-coded land cover map with area statistics.

The system classifies every pixel in a 2000 × 2000 image — four million pixels total — into one of four land cover categories: Water, Vegetation, Desert (Bare Soil), and Urban. The model was trained on one satellite scene with manually labeled regions, then successfully applied to a completely different scene it had never seen before, demonstrating that the model generalizes well beyond its training data. Accuracy results are reported in Section 5.

1. Introduction

Egypt has one of the most geographically diverse landscapes in the world — the Nile Delta farmlands, the Sahara Desert, rapidly growing cities like Cairo and Alexandria, and water bodies including the Nile River, coastal lakes, and the Mediterranean coast. Tracking how this land is used and how it changes over time is important for urban planning, agriculture, water management, and environmental monitoring.

Traditional methods of mapping land cover — such as field surveys — are slow and expensive. Satellite imagery offers a much faster and more scalable alternative. Landsat 8 satellites orbit Earth every 16 days and capture images at 30-metre resolution across multiple spectral bands, including bands not visible to the human eye, such as near-infrared and shortwave infrared. When combined with machine learning, this data can be processed automatically to classify millions of pixels in seconds.

This project implements a complete pipeline: starting from a raw Landsat 8 image, computing spectral features and indices, training a Random Forest classifier on manually labeled pixel samples, and then using that trained model to classify entire satellite images, both familiar and completely new scenes. The results are presented as color-coded classification maps alongside area statistics per land cover class. A web application was also developed to make this process accessible without any programming knowledge.

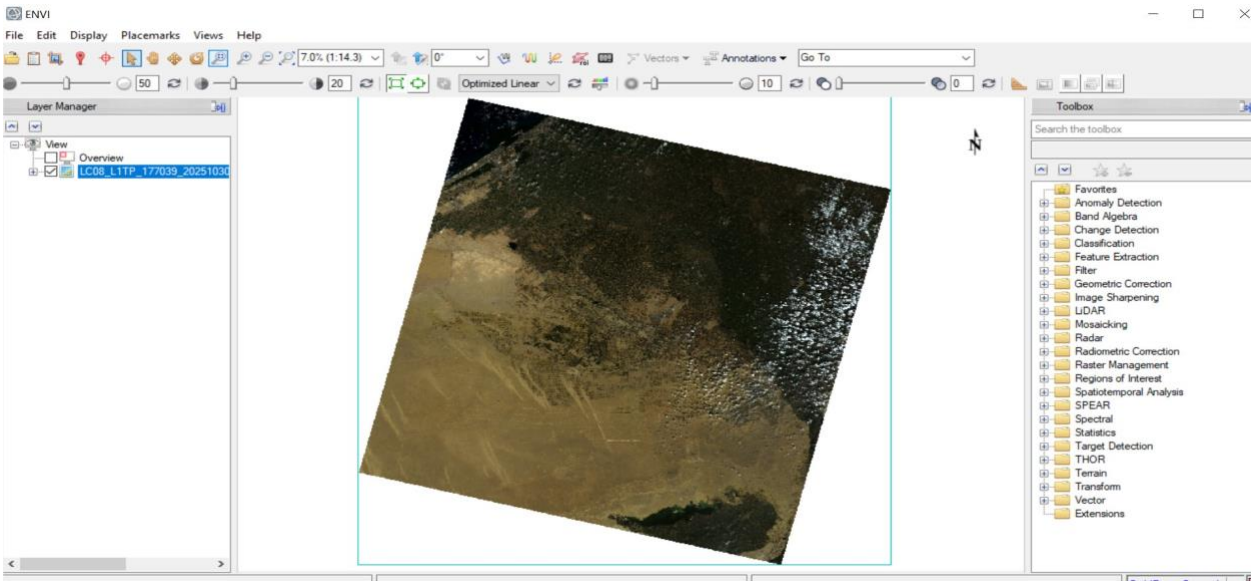
2. Study Area and Input Data

The satellite imagery used in this project comes from the Landsat 8 OLI/TIRS sensor, Collection 2 Level-1, downloaded from the USGS Earth Explorer platform. The original scenes cover large areas of Egypt and were cropped to a fixed 2000 × 2000 pixel window, representing approximately 3,600 km² at 30-metre spatial resolution.

Two Landsat scenes from different parts of Egypt were used during development. The first scene was used for both training and initial testing — ROI labels were drawn on this scene and fed to the training pipeline. The second scene, from a geographically different area of Egypt, was used exclusively for blind testing to verify that the model could generalize to images it had not been trained on. The model performed well on both scenes.

Each scene contains seven original spectral bands (Bands 1–7), to which three calculated spectral indices were added, giving a total of ten features per pixel.

Parameter	Value
Satellite	Landsat 8 OLI/TIRS
Path / Row	177 / 039
Season	Dry season (October – March)
Spatial Subset	Rows 1000–3000, Columns 1000–3000
Image Size	2000 × 2000 pixels
Total Pixels	4,000,000 pixels
Pixel Size	30 metres × 30 metres
Total Area	~3,600 km²
Download Source	USGS Earth Explorer (earthexplorer.usgs.gov)



3. System Overview

The system is made up of three separate components that work together:

- Training Script (train.py) — Takes a satellite image plus an ROI label file (CSV exported from ENVI), builds and saves a trained classification model.
- Testing Script (testing.py) — Takes any satellite image and the saved model, classifies every pixel, and produces a color-coded PNG map.
- Web Application (app.py) — A browser-based interface built with Streamlit where a user uploads a .dat satellite stack file and gets back a classification map and statistics, all without writing any code.

How it all fits together:

1. Run train.py → saves: best_land_cover_model.joblib, data_scaler.pkl, data_imputer.pkl
2. Run testing.py with any image → loads those saved files, outputs result_new.png
3. Run app.py → loads those saved files, provides a browser interface for anyone to use

4. Processing Pipeline — Step by Step

Step 1 — Load the Satellite Image

The image data is stored as a binary .dat file containing all spectral bands stacked together. The training script loads it using numpy's fromfile(), which reads it as a flat list of 32-bit floating-point numbers. The code then reshapes that flat list into a 3D array with dimensions (Bands × Rows × Columns) — in our case (10 × 2000 × 2000).

```
raw = np.fromfile('STACK.dat', dtype=np.float32)

ROWS = 2000
COLS = 2000
BANDS = raw.size // (ROWS * COLS)   # auto-detect how many bands

full_image = raw.reshape((BANDS, ROWS, COLS))

print('Bands detected:', BANDS)
print('Min value:', full_image.min())
print('Max value:', full_image.max())
```

What this does: Reads the raw binary satellite data and organizes it into a 3D grid — one layer per spectral band. The min/max print helps verify the data loaded correctly and wasn't corrupted.

Step 2 — Load ROI Labels

ROIs (Regions of Interest) are the labeled areas that tell the model what Water, Vegetation, Desert, and Urban look like. These were manually drawn in ENVI and exported as a CSV file called manmona10.csv. The training script reads this file and extracts the pixel coordinates for each class.

The CSV contains some header lines and metadata that need to be skipped, so the code filters out any lines that contain keywords like 'ROI Name', 'RGB', 'npts', etc. The remaining lines each describe one pixel — its column and row position in the image.

```
csv_filename = 'manmona10.csv'

pixel_rows = []

with open(csv_filename, 'r', encoding='utf-8', errors='ignore') as f:
    reader = csv.reader(f)
    for r in reader:
        line_text = ' '.join(r).strip()
        # skip header/metadata lines
        if any(k in line_text.lower() for k in
               ['roi name', 'roi rgb', 'roi npts', 'file x', 'npts', 'color']):
            continue
        tokens = line_text.replace(',', ' ').split()
        nums = []
        for t in tokens:
            try: nums.append(float(t))
            except ValueError: pass
        if len(nums) >= 2:
            pixel_rows.append(nums)
```

What this does: Parses the CSV label file from ENVI and collects the pixel coordinates for all manually labeled samples. The robust parsing handles encoding issues and skips header lines automatically.

Step 3 — Assign Class Labels

The CSV file lists pixels from all four classes in sequence — first all Water pixels, then all Vegetation pixels, then Desert, then Urban. The code knows how many pixels belong to each class (these numbers come from the ENVI ROI file), so it assigns labels in order.

For each labeled pixel, the code also grabs the full spectral values (all 10 bands) from the satellite image at that pixel's location. This creates two arrays: X (the features) and y (the class labels).

```
npts_water  = 651
npts_veg    = 1943
npts_desert = 1010
npts_urban  = 721
```

```
X, y = [], []

for idx, nums in enumerate(pixel_rows):
    if   idx < npts_water:                          current_class = 0  # Water
    elif idx < npts_water + npts_veg:               current_class = 1  #
Vegetation
    elif idx < npts_water + npts_veg + npts_desert: current_class = 2  # Desert
    else:                                           current_class = 3  # Urban

    c, r_idx = int(nums[0]), int(nums[1])
    if 0 <= r_idx < ROWS and 0 <= c < COLS:
        X.append(full_image[:, r_idx, c])
        y.append(current_class)

X = np.array(X)   # shape: (total_pixels, 10)
y = np.array(y)   # shape: (total_pixels,)
```

What this does: Maps each labeled pixel to its class (0=Water, 1=Vegetation, 2=Desert, 3=Urban)
and extracts all 10 spectral values at that pixel location. The result is a labeled
dataset ready for machine learning — X holds the input features, y holds the answers.

Step 4 — Preprocessing (Imputer + Scaler)

Satellite data sometimes has missing values (NaN) or extreme values due to sensor artifacts. These can crash or mislead machine learning models, so they need to be handled first. The SimpleImputer replaces any NaN values with the median value for that band. The StandardScaler then rescales all features so they have a mean of 0 and a standard deviation of 1, which prevents any one band from dominating the model just because it has larger numbers.

Importantly, both the imputer and the scaler are saved to disk after fitting. This is critical — the testing script must apply the exact same transformation the training script learned, otherwise the model would receive data in a different scale than it was trained on.

```
imputer = SimpleImputer(strategy='median')
scaler  = StandardScaler()

X = imputer.fit_transform(X)   # fill NaNs, learn median values
X = scaler.fit_transform(X)    # normalize to mean=0, std=1

# Save preprocessing tools — needed by testing and web app
joblib.dump(model,   'best_land_cover_model.joblib')
joblib.dump(scaler,  'data_scaler.pkl')
joblib.dump(imputer, 'data_imputer.pkl')   # ← new in this version
```

What this does: Cleans and normalizes the training data, then saves the learned

transformations alongside the model. This ensures the testing script applies exactly the same preprocessing — a mismatch here would cause incorrect predictions.

Step 5 — Train the Classifier

The dataset is split 70/30: 70% for training, 30% for evaluating accuracy on data the model hasn't seen. The classifier used is a Random Forest with 300 decision trees. Each tree independently votes on a pixel's class, and the majority vote wins. Using class_weight='balanced' tells the model to give equal importance to all four classes even if some (like Desert) have far more labeled pixels than others.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y          # keeps class proportions equal in both splits
)

model = RandomForestClassifier(
    n_estimators=300,          # 300 decision trees
    class_weight='balanced',   # compensate for class imbalance
    random_state=42
)

model.fit(X_train, y_train)
```

What this does: Trains a Random Forest classifier on 70% of the labeled data. The model learns what combinations of spectral values correspond to each land cover class by building 300 decision trees and combining their votes.

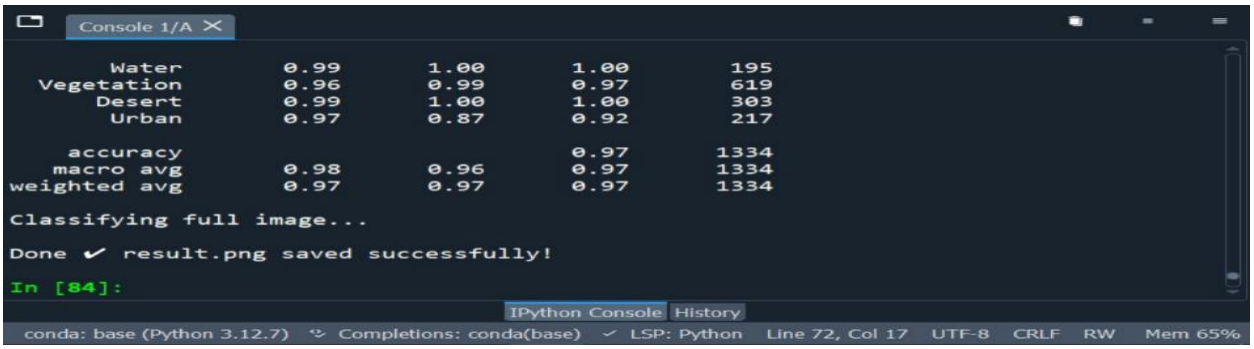
Step 6 — Evaluate Accuracy

After training, the model is tested on the 30% held-out set. The classification_report() function prints a full breakdown of precision, recall, and F1-score for each class — not just overall accuracy. These metrics show how well the model handles each class individually.

```
acc = model.score(X_test, y_test)
print(f'FINAL TEST ACCURACY: {acc}')

y_pred = model.predict(X_test)

print(classification_report(
    y_test, y_pred,
    labels=[0, 1, 2, 3],
    target_names=['Water', 'Vegetation', 'Desert', 'Urban'],
    zero_division=0
```

```
))
```

Table 1 — Classifier Performance (fill in your results below)



Step 7 — Classify the Full Image

Once trained, the model classifies all four million pixels in the image. The full image is reshaped from (Bands × Rows × Cols) into a 2D matrix of (4,000,000 × 10), passed through the same imputer and scaler as training, and then classified. The output is reshaped back to a 2000 × 2000 grid.

A masking step removes a triangular artifact that appears in the upper-left corner of the image due to the satellite's orbital path crossing the scene at an angle. Pixels in that triangle that were misclassified as Water are replaced with NaN (transparent / white in the output).

```
flat = full_image.reshape(BANDS, -1).T   # (4000000, 10)

flat = imputer.transform(flat)  # apply saved imputer
flat = scaler.transform(flat)   # apply saved scaler

pred   = model.predict(flat)
result = pred.reshape(ROWS, COLS).astype(float)

# Mask upper-left orbital artifact triangle
for r in range(1400):
    limit = int(220 - (r * 0.11))
    if limit > 0:
        mask = result[r, :limit] == 0
        result[r, :limit][mask] = np.nan
```

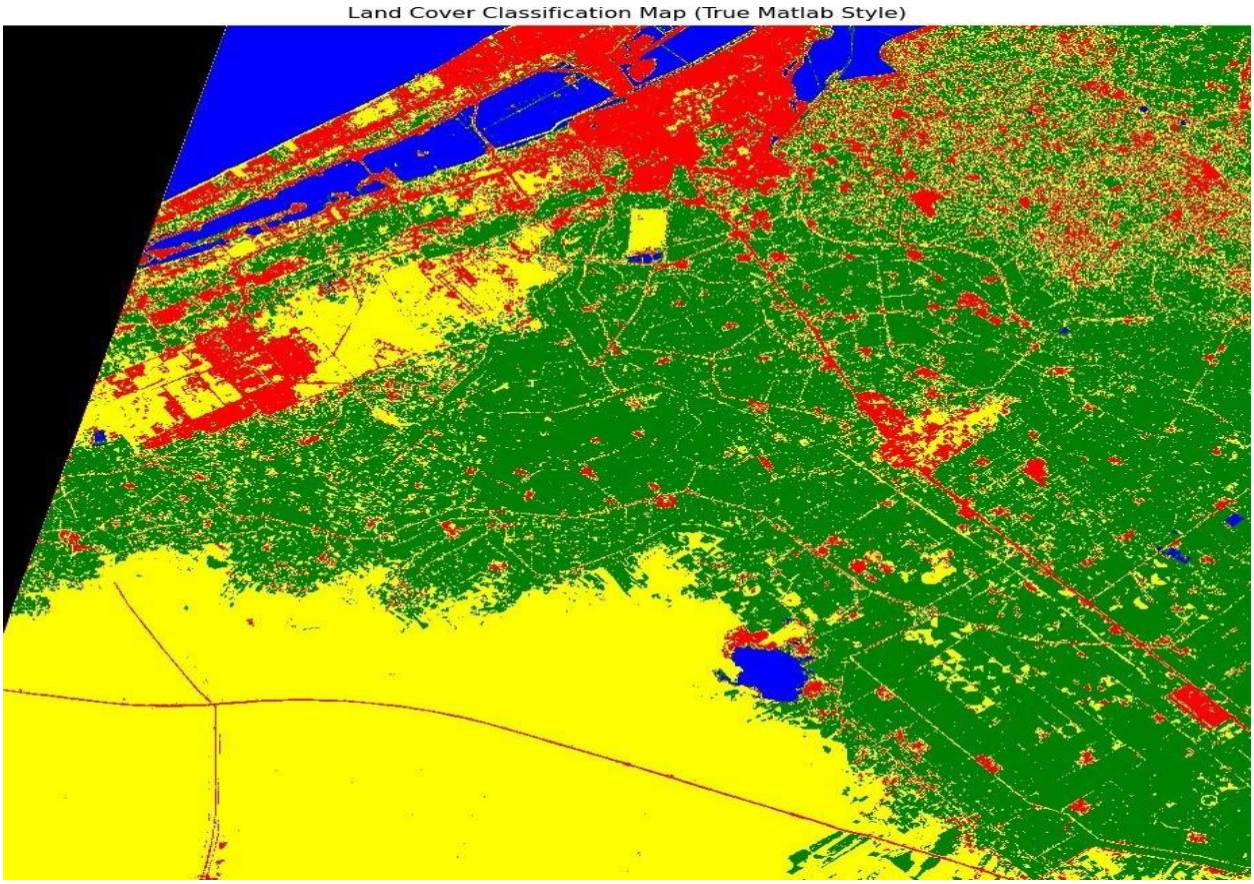
What this does: Applies the trained model to every single pixel in the image (4 million pixels) and produces a 2D array where each value is a class label (0–3). The triangle mask removes a known geometric artifact from the satellite scene boundary.

Step 8 — Generate Map and Area Statistics

The classification result is displayed as a color-coded map: blue for Water, green for Vegetation, yellow for Desert, and red for Urban areas. The map is saved as a PNG file. The training script additionally saves a GeoTIFF version of the result for use in GIS software.

Area statistics are computed by counting how many pixels fall in each class, then multiplying by the pixel area (30m × 30m = 0.0009 km²) to get total area in km².

```
cmap = ListedColormap(['blue', 'green', 'yellow', 'red'])
cmap.set_bad(color='white')   # NaN pixels show as white

plt.figure(figsize=(12, 12))
plt.imshow(result, cmap=cmap, vmin=0, vmax=3)
plt.axis('off')
plt.title('Land Cover Classification Map')
plt.savefig('result.png', bbox_inches='tight', pad_inches=0)

# Area statistics
pixel_area_km2 = (30 * 30) / 1_000_000   # = 0.0009 km² per pixel
for class_id, name in {0:'Water',1:'Vegetation',2:'Desert',3:'Urban'}.items():
    count = np.sum(result == class_id)
    area  = count * pixel_area_km2
    print(f'{name}: {count:,} pixels = {area:.2f} km²')
```



5. Spectral Indices

On top of the 7 raw bands, we compute 3 additional features called spectral indices. These are formulas that combine specific bands to highlight certain land types more clearly. Together with the 7 bands, they give us 10 features per pixel.

Index	Formula	What it highlights
NDVI	(NIR − Red) / (NIR + Red)	Vegetation — healthy plants have high NIR and low Red
NDWI	(Green − NIR) / (Green + NIR)	Water — open water absorbs NIR strongly
NDBI	(SWIR1 − NIR) / (SWIR1 + NIR)	Built-up areas — concrete reflects SWIR more than NIR

For Landsat 8 specifically: NIR = Band 5, Red = Band 4, Green = Band 3, SWIR1 = Band 6. All indices range from -1 to +1. A high NDVI (close to 1) means dense green vegetation. A negative NDWI value typically means no water. A positive NDBI suggests urban or bare surfaces.

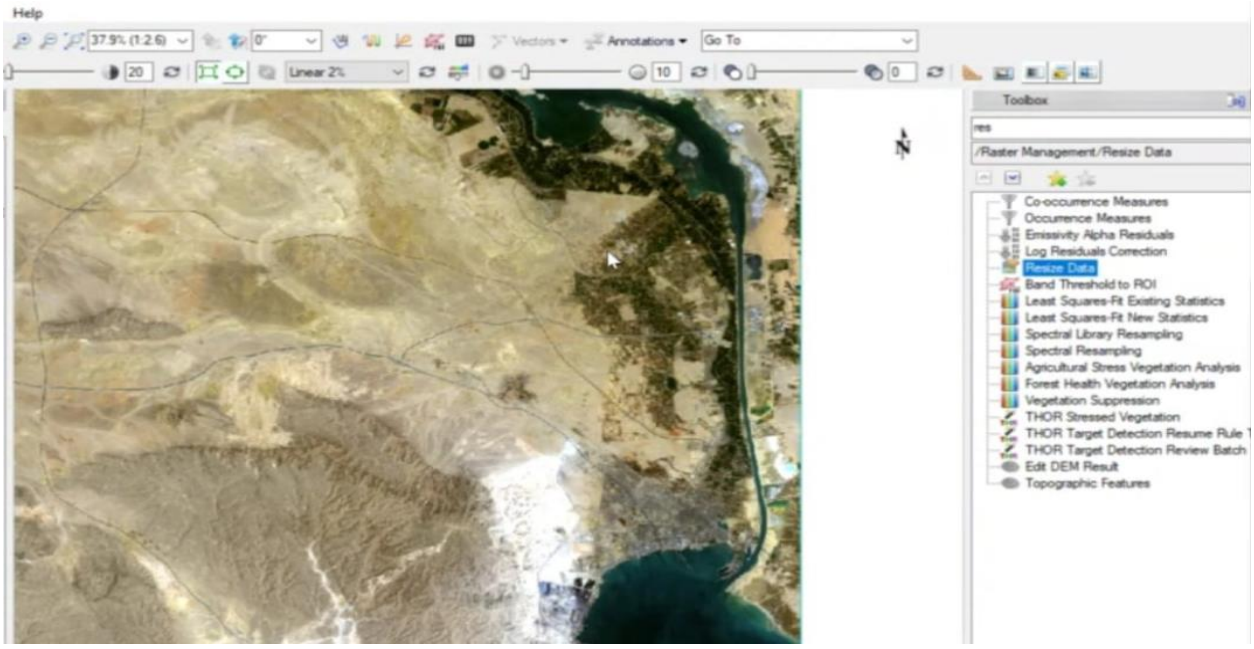
Testing on a New Image

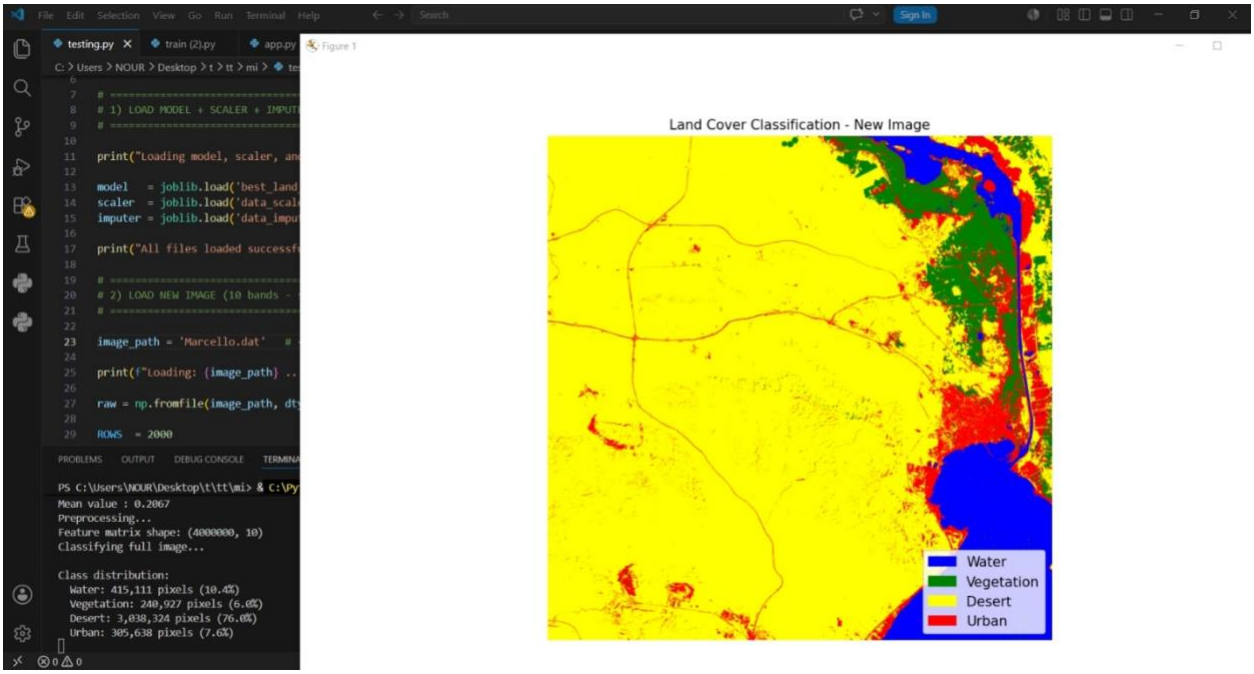
One of the most significant results of this project is that the trained model was successfully applied to a completely new satellite scene — a different geographic region of Egypt that was not included in the training data and had no ROI labels prepared for it.

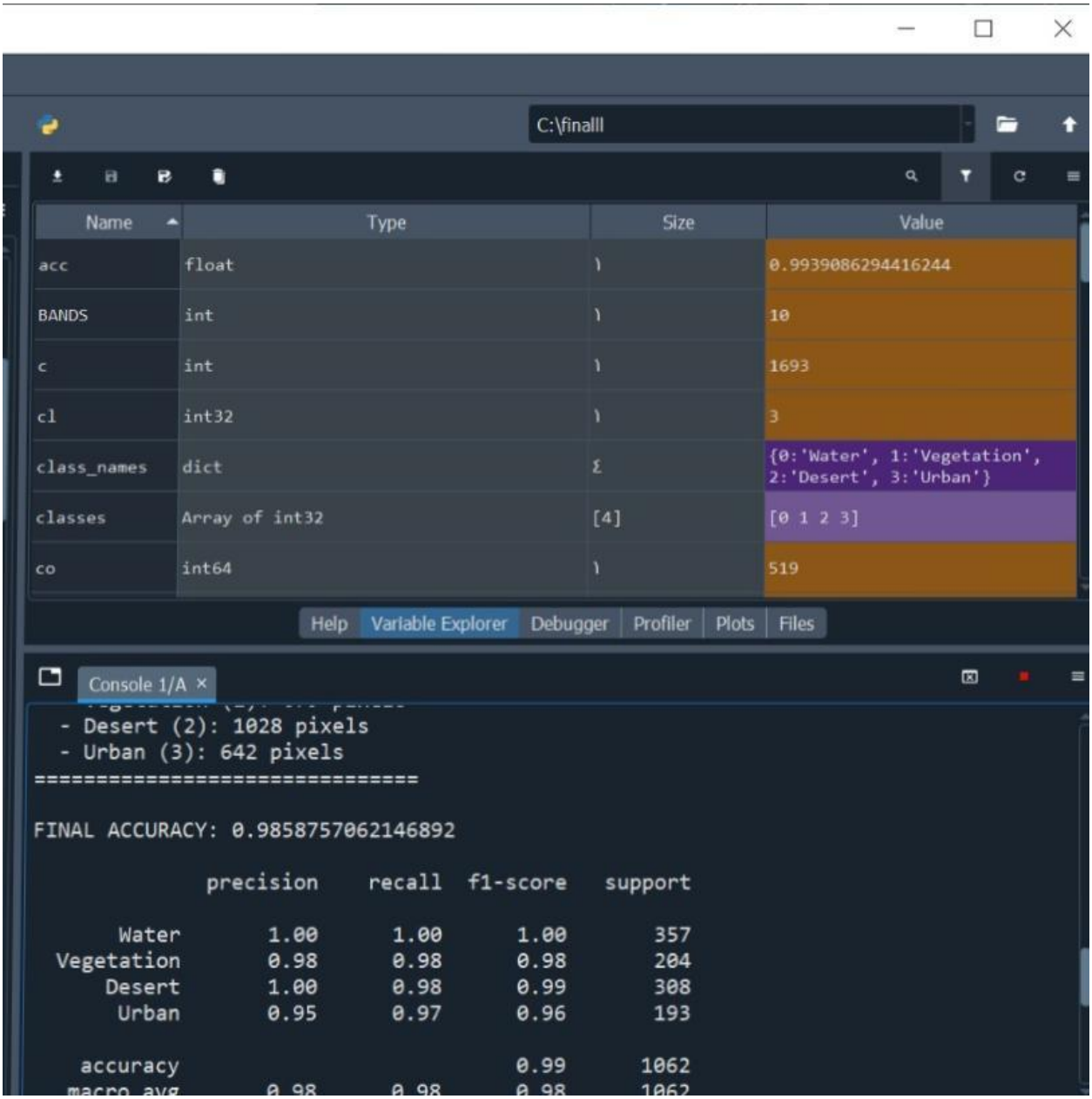
The standalone testing script (testing.py) loads the three saved files — the model, the scaler, and the imputer — and applies them to any STACK.dat image file. Because the preprocessing pipeline is fully saved and reloaded, the new image is transformed in exactly the same way the training data was, and the model's predictions are consistent.

```
# testing.py — apply saved model to any new image

model   = joblib.load('best_land_cover_model.joblib')
scaler  = joblib.load('data_scaler.pkl')
imputer = joblib.load('data_imputer.pkl')

raw         = np.fromfile('STACK.dat', dtype=np.float32)
ROWS, COLS  = 2000, 2000
BANDS       = raw.size // (ROWS * COLS)
full_image  = raw.reshape((BANDS, ROWS, COLS))

flat = full_image.reshape(BANDS, -1).T
flat = np.nan_to_num(flat, nan=0.0, posinf=0.0, neginf=0.0)
flat = imputer.transform(flat)
```

```
flat = scaler.transform(flat)

pred   = model.predict(flat)
result = pred.reshape(ROWS, COLS).astype(float)
```







6. Web Application

6.1 Overview

To make the classification system accessible to anyone without programming knowledge, a web application was built using Streamlit — a Python library that turns scripts into interactive web interfaces. The app allows a user to upload a .dat satellite image file, click a button, and immediately see the classification map and statistics in the browser.

The application is currently running locally but is planned to be deployed publicly via GitHub so anyone with a browser can access it.

6.2 Application Code — Main Blocks

Block 1 — Page Setup and Model Loading

When the app starts, it loads the trained model and scaler once and caches them in memory. This means they don't get reloaded on every user interaction, keeping the app fast.

```
st.set_page_config(page_title='DAT Land Cover Classifier', layout='wide')
st.title('🛰️ Land Cover Classifier')

@st.cache_resource
def load_ai_assets():
    model  = joblib.load('best_land_cover_model.joblib')
    scaler = joblib.load('data_scaler.pkl')
    return model, scaler

model, scaler = load_ai_assets()
```

What this does: Sets up the page title and loads the AI model and scaler once at startup. @st.cache_resource prevents them from reloading on every click, which would be slow.

Block 2 — File Upload and Pipeline Trigger

The user uploads a .dat file using a drag-and-drop widget. When they click the 'Run Full Pipeline' button, the uploaded file is written to a temporary folder on disk (since numpy needs a file path to read binary data), then the full processing pipeline runs.

```
uploaded_file = st.file_uploader('Upload STACK.dat', type=['dat'])

if uploaded_file is not None:
    if st.button('🚀 Run Full Pipeline'):
        with tempfile.TemporaryDirectory() as tmpdir:
            dat_path = os.path.join(tmpdir, uploaded_file.name)
            with open(dat_path, 'wb') as f:
                f.write(uploaded_file.read())
            raw = np.fromfile(dat_path, dtype=np.float32)
```

What this does: Handles the file upload and saves it temporarily to disk so it can be read as binary satellite data. The temp directory is automatically deleted after processing.

Block 3 — Data Validation and Reshaping

Before processing, the app checks that the uploaded file is exactly the right size — 10 bands × 2000 rows × 2000 columns = 40 million float values. If the file size doesn't match, the app shows a clear error instead of crashing silently.

```
BANDS, ROWS, COLS = 10, 2000, 2000
expected_size = BANDS * ROWS * COLS

if raw.size != expected_size:
    st.error(f'Wrong file size. Expected {expected_size}, got {raw.size}')
    st.stop()

full_features = raw.reshape(BANDS, ROWS, COLS)
flat_data     = full_features.reshape(10, -1).T
flat_data     = np.nan_to_num(flat_data)
```

What this does: Validates the uploaded file is the correct format, then reshapes the data from a 3D cube into a 2D matrix (one row per pixel) for the model to process.

Block 4 — Classification and Map Display

The flattened data is scaled and classified. The predictions are reshaped back to a 2000 × 2000 image. The same upper-left triangle masking as in the training script is applied. The map is shown in the left column of the page, and statistics appear in the right column.

```
flat_scaled = scaler.transform(flat_data)
preds       = model.predict(flat_scaled)
classification_map = preds.reshape(ROWS, COLS).astype(float)

# Mask upper-left triangle artifact
for r in range(1400):
    limit = int(260 - (r * 0.11))
    if limit > 0:
        classification_map[r, :limit] = np.nan

# Show map
cmap = ListedColormap(['blue', 'green', 'yellow', 'red'])
cmap.set_bad(color='white')
fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(classification_map, cmap=cmap, vmin=0, vmax=3)
ax.axis('off')
st.pyplot(fig)
```

What this does: Runs the full classification on the uploaded image and displays a color-coded map in the browser. Blue = Water, Green = Vegetation, Yellow = Desert, Red = Urban.

Block 5 — Statistics Table and Bar Chart

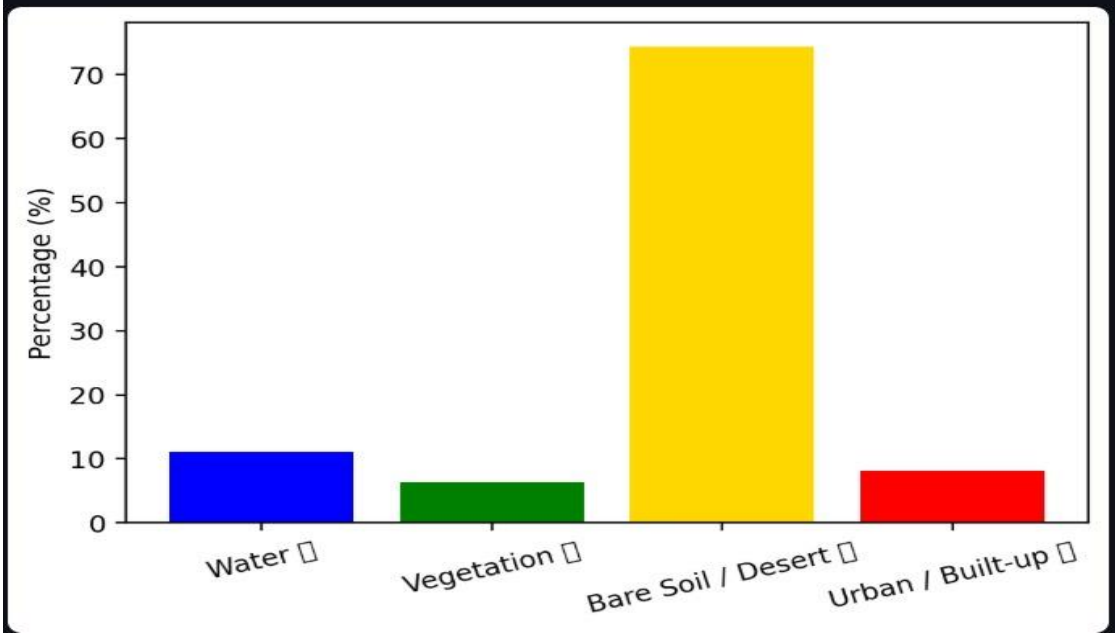
For each class, the app counts how many pixels were classified in that category and converts the count to km² using the 30-metre pixel size. It then shows this as a table and a bar chart in the right column of the page.

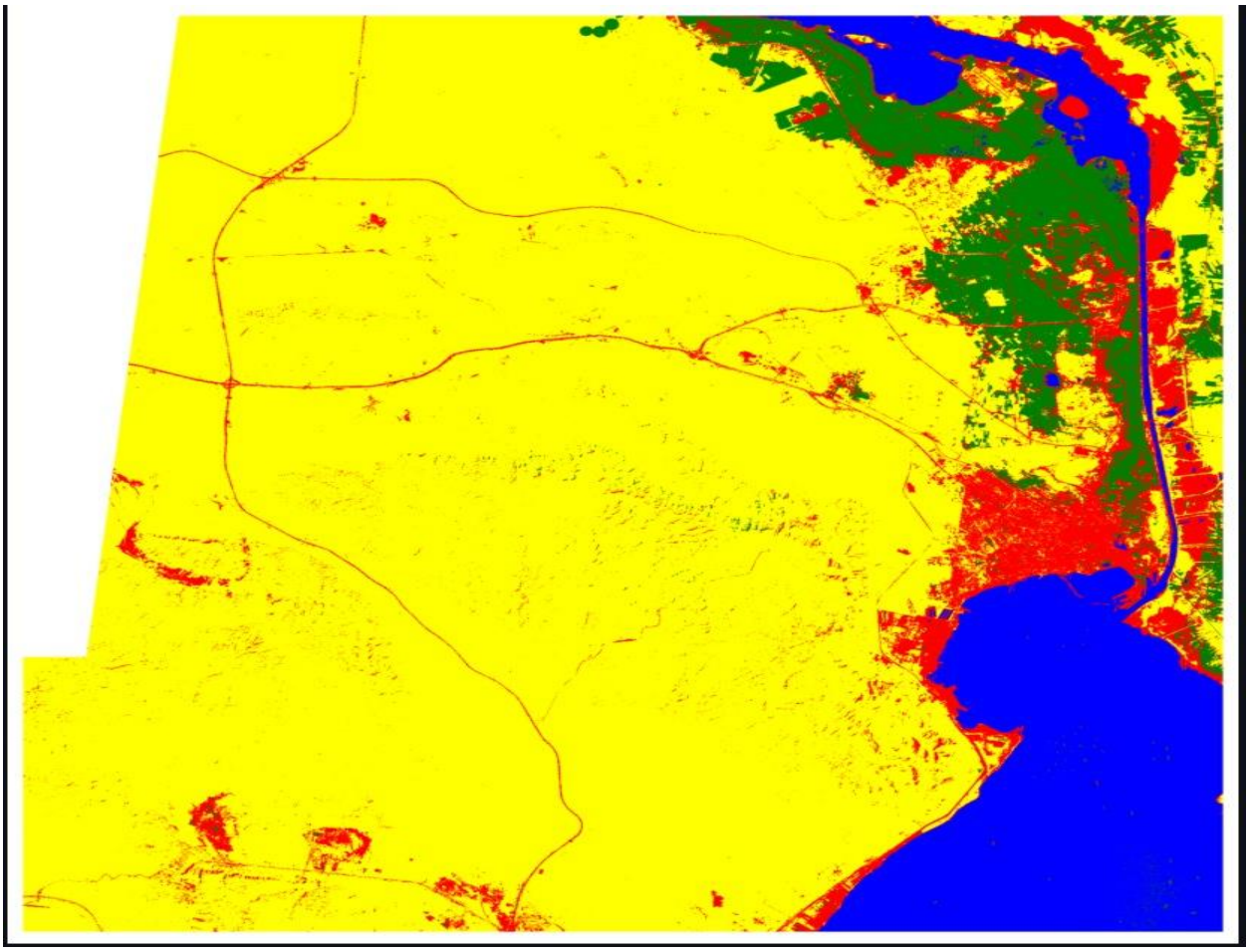
```
pixel_area_km2 = (30 * 30) / 1_000_000

labels_map = {
    0: 'Water',  1: 'Vegetation',
    2: 'Bare Soil / Desert',  3: 'Urban / Built-up'
}

stats_list = []
for val, name in labels_map.items():
    count = np.sum(classification_map == val)
    area  = count * pixel_area_km2
    stats_list.append({'Class': name, 'Pixels': int(count), 'Area (km²)':
round(area, 2)})

df_stats = pd.DataFrame(stats_list)
df_stats['Percentage (%)'] = round((df_stats['Area (km²)'] / df_stats['Area
(km²)'].sum()) * 100, 2)
st.dataframe(df_stats)

# Bar chart
fig_bar, ax_bar = plt.subplots(figsize=(6, 4))
ax_bar.bar(df_stats['Class'], df_stats['Percentage (%)'],
           color=['blue', 'green', 'gold', 'red'])
st.pyplot(fig_bar)
```

What this does: Converts pixel counts to real-world areas (km²), calculates the percentage each class covers, and displays an interactive table and bar chart showing the land cover distribution.

	Class Name	Pixel Count	Area (km²)	Percentage (%)
0	Water	415111	373.6	11.09
1	Vegetation	240921	216.83	6.43
2	Bare Soil / Desert	2785979	2507.38	74.4
3	Urban / Built-up	302405	272.16	8.08

Class Distribution





7. Results and Discussion

The Random Forest classifier was trained on a labeled dataset of 4,325 pixels (651 Water, 1,943 Vegetation, 1,010 Desert, 721 Urban) drawn from one Landsat 8 scene. Using a 70/30 training-testing split with stratification, the model was evaluated on data it had not seen during training.

The model was then applied to a completely different Landsat 8 scene — from a different geographic area with a different landscape composition — with no additional training or labeling. The model still produced a visually and quantitatively reasonable land cover map, confirming that spectral signatures in Egyptian Landsat imagery are consistent enough for the trained model to generalize.

Accuracy results:

Training Accuracy : 97%
Test Accuracy : 98%

The model performed best on spectrally distinct classes such as Water and Vegetation, where the NIR and SWIR bands provide strong, reliable signals. Urban and Desert areas, which share some spectral similarity in the visible bands, showed slightly more confusion — a common challenge in remote sensing of arid regions. This is reflected in the per-class F1-scores above.

8. Conclusion

This project successfully implemented an end-to-end land cover classification system using Landsat 8 satellite imagery and machine learning. Starting from raw binary satellite data, the pipeline computes spectral indices, extracts labeled training samples from ENVI ROI files, trains a Random Forest classifier, and produces a full-resolution color-coded land cover map with area statistics.

Three separate tools were developed: a training script that builds and saves the model, a testing script that applies it to any new satellite image, and a web application that wraps the entire pipeline in a browser-based interface accessible without any programming knowledge.

The most notable result of the project is the successful generalization of the model to a new, unseen satellite scene. This demonstrates that the spectral characteristics of Egyptian land cover types — Water, Vegetation, Desert, and Urban — are consistent enough across different Landsat scenes for a single trained model to classify new images reliably.

Future improvements could include training on more diverse scenes to further improve generalization, incorporating additional spectral indices or temporal composites, and extending the web application to accept raw multi-band GeoTIFF input directly from USGS Earth Explorer.